Laplacian Eigenvector Tokenization

Giving graph nodes a coordinate system the way sines and cosines give one to a line

positional encoding · `features.laplacian_positional_encoding` · 2026-06-21

In one sentence. A transformer needs to tell its tokens apart, but a graph's node numbers are arbitrary. Laplacian eigenvectors hand each node a short vector of coordinates that depend only on **where it sits in the graph's shape** — not on how it was numbered — so two relabelled copies of the same graph get the same encoding.

1 Why a graph needs positional encoding at all

A transformer sees a **set of tokens** and lets every token attend to every other. By itself, attention is permutation-invariant: shuffle the tokens and you get the same answer shuffled. For text that is wrong — "dog bites man" $\neq$ "man bites dog" — so transformers add a **positional encoding**: a vector glued to each token that says **where it is**.

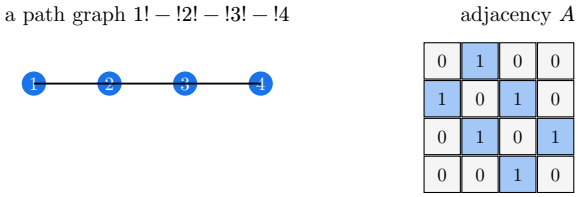
For a sentence "where" is easy — position $1, 2, 3, \ldots$ along a line, encoded with sines and cosines of different frequencies. For a graph there is no such line. A node has no natural index; the only thing that is real is **the pattern of connections around it**. So the question becomes:

What is the graph-shaped analogue of "sine and cosine of position"?

The answer is the **eigenvectors of the graph Laplacian**. They are, quite literally, the natural vibration modes of the graph — the graph's own sines and cosines.

2 Building block 1 — the Laplacian matrix

Start from two matrices you already know. For a graph on n nodes:



Adjacency A $A_{ij} = 1$ if nodes i and j share an edge, else 0. It records **who connects to whom**.
Degree D a diagonal matrix, $D_{ii} = \sum_j A_{ij}$ = the number of edges at node i. It records **how busy each node is**.

The **combinatorial Laplacian** is simply their difference:

$$L = D - A$$

That tiny formula is doing something specific. Apply L to a vector x that assigns a number x_i to each node, and look at row i:

$$(Lx)_i = D_{ii}x_i - \sum_{j\sim i} x_j = \sum_{j\sim i}(x_i - x_j)$$

So $(Lx)_i$ measures **how different node i's value is from its neighbours'** — a discrete second derivative, the graph version of $-\nabla^2$. This is why L is called the Laplacian: on a grid it literally becomes the finite-difference Laplace operator.

2.1 The quadratic form: L measures smoothness

The single most important fact about L is what it computes in the quadratic form $x^\top Lx$:

$$x^\top L x = \sum_{(i,j)\in E} (x_i - x_j)^2$$

Read this slowly. Hand the graph any assignment of numbers to nodes. Then $x^\top L x$ is the **total squared disagreement across edges**. It is large when neighbours hold very different values (a jagged signal) and small when neighbours agree (a smooth signal). L is the graph's built-in **roughness meter**.

Because it is a sum of squares, $x^\top L x \geq 0$ always: L is **positive semi-definite**. Combined with L being real and symmetric, this guarantees everything we need in the next step.

3 Building block 2 — eigenvectors as vibration modes

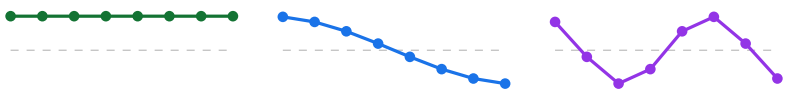
Because L is real, symmetric, and PSD, the spectral theorem gives us a full set of n real eigenvectors $u_1, ..., u_n$ that are mutually orthogonal, with non-negative eigenvalues

$$0 = \lambda_1 \leq \lambda_2 \leq ... \leq \lambda_n.$$

Each eigenvector solves $Lu = \lambda u$. Rearranged through the quadratic form, the eigenvalue **is** the roughness of its eigenvector:

$$\lambda_k = u_k^\top L u_k = \sum_{(i,j)\in E} (u_k(i) - u_k(j))^2 \quad (\|u_k\| = 1).$$

So eigenvectors are ordered **from smoothest to roughest**. This is exactly the physics of a vibrating string or drum: low modes wobble slowly across the whole object, high modes oscillate rapidly. On a path graph the eigenvectors are literally cosine waves of increasing frequency:



$\lambda_1 = 0$ · constant λ_2 · 1 oscillation λ_k · many oscillations

The leftmost mode is flat — every node gets the same value. The next splits the graph gently into "one side high, one side low". Higher modes carve the graph into ever finer alternating regions. **These alternating regions are the coordinates we are after.**

4 Why the first eigenvector is thrown away

The smallest eigenvalue is always $\lambda_1 = 0$, and for a connected graph its eigenvector is the **constant** vector $u_1 = (1, 1, ..., 1)/\sqrt{n}$. Check it: $L \cdot \mathbf{1} = (D - A)\mathbf{1} = 0$ because each row of A sums to that node's degree.

A constant assigns **the same coordinate to every node**, so it tells you nothing about position. That is why the implementation skips it:

```
# features.py — symmetric normalized Laplacian, drop the trivial mode
_, eigvecs = torch.linalg.eigh(L)   # ascending eigenvalues
pe = eigvecs[:, 1:k + 1]            # skip column 0 (constant), keep next k
```

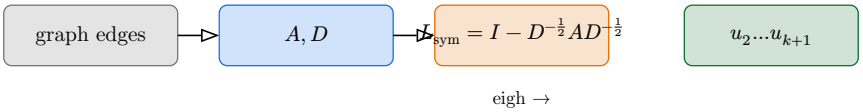
Aside — counting connected components. The multiplicity of eigenvalue 0 equals the number of connected components. If the graph splits into two pieces there are **two** independent constant-like modes (one per piece). This is the spectral fingerprint of connectivity — the very property the connectedness task is about — and a reason the Laplacian spectrum is a natural language for these experiments. See [[connectedness-dataset-leak]].

5 The normalized Laplacian we actually use

`features.py` does not use $L = D - A$ directly. It uses the **symmetric normalized** Laplacian:

$$L_{\text{sym}} = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}.$$

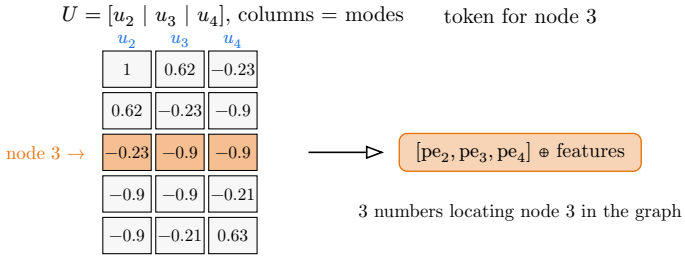
The normalization rescales each node by $1/\sqrt{\deg}$, which keeps high-degree hubs from dominating the spectrum and pins all eigenvalues into the clean range $\lambda \in [0, 2]$. It is the same operator that sits inside a GCN layer, so the positional encoding speaks the same dialect as message passing.



The rest of the story — skip u_1, keep the next k — is unchanged. The output is an $n \times k$ matrix: ***k* structural coordinates per node**.

6 From eigenvectors to tokens

Stack the chosen eigenvectors as columns. Reading the matrix **by rows** gives, for each node, its coordinates in the graph's natural basis:



These k numbers are concatenated onto (or added to) each node token before it enters the transformer. Now attention can distinguish nodes **by their structural position**, and an edge token can tell its two endpoints apart by their coordinates — without ever consulting an arbitrary node index.

6.1 Why this is the encoding the other failures were missing

The companion note on tokenization (`tokenization.typ`) ended with four properties a good node identity must have. Laplacian PE hits all four:

unique within a graph	distinct nodes almost surely get distinct coordinate vectors,
stable across epochs	computed once from the graph, never trained — no gradient fight,
not shared across graphs	coordinates come from this graph's own spectrum, not a global table,
structure-only	depends only on edges, so a relabelled copy gets the identical encoding.

That last point is the crucial one and deserves a proof sketch.

6.2 Permutation equivariance — the property we actually wanted

Relabel the nodes with a permutation P. Then $A \mapsto PAP^\top$ and $L \mapsto PLP^\top$. If $Lu = \lambda u$, then

$$(PLP^\top)(Pu) = PLu = \lambda(Pu),$$

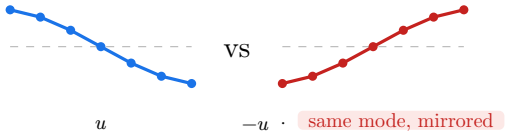
so the eigenvectors simply get **permuted the same way the nodes did**. Node i's coordinates travel with node i no matter what number you paint on it. This is exactly the **structure-only** property that adjacency-row tokens lacked — there, relabelling produced completely different tokens for the same shape.

7 The one real catch: sign and basis ambiguity

Eigenvectors are not perfectly unique, and this is the thing that trips people up.

7.1 Sign flips

If u is a unit eigenvector, so is $-u$ — they solve $Lu = \lambda u$ equally well. The solver picks a sign **arbitrarily**, and it may pick differently for the same graph on another run or another machine.



Both encode the identical structure, but a model that memorized "$\text{pe}_2 > 0$ means left side" breaks when the sign flips. Standard fixes:

Random sign flipping during training, multiply each eigenvector by a random ± 1. The model is forced to learn features invariant to sign — the cheap, common default.

Sign-invariant networks feed both u and $-u$ through a shared net and combine symmetrically (SignNet), so the output cannot depend on the choice.

7.2 Repeated eigenvalues

When $\lambda_k = \lambda_{k+1}$ the eigenvectors are only defined **up to rotation within that eigenspace** — there is no canonical choice at all, not even up to sign. Highly symmetric graphs (cycles, grids) have many such repeats. This is a deeper ambiguity than sign flips and is the main theoretical wrinkle of Laplacian PE; sign-flip augmentation handles the common case, and basis-invariant architectures handle the rest.

Practical defaults baked into `features.py`. (1) Use L_{sym} so eigenvalues live in $[0, 2]$. (2) Always skip u_1. (3) Keep a small k (the lowest, smoothest modes carry the most global structure); pad with zeros when the graph has fewer than $k+1$ nodes. (4) Apply random sign flips at train time. With these four habits, Laplacian PE is a robust, permutation-aware coordinate system for graph tokens.

8 A worked micro-example

Take the path $1! - !2! - !3$. Its Laplacian and the two non-trivial modes:

$$L = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}, \quad u_3 = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}.$$

Read the rows to get each node's 2-D coordinate (we keep $k = 2$):

node	$\text{pe}_1 = u_2$	$\text{pe}_2 = u_3$	**reading**
1	$+0.71$	$+0.41$	an end node — extreme on the slow mode
2	0.00	-0.82	the centre — zero on u_2, the dip of u_3
3	-0.71	$+0.41$	the other end — mirror of node 1

Node 2 sits at the **centre of mass** of the slow mode ($\text{pe}_1 = 0$) while the two ends are pushed to opposite extremes — the encoding has discovered the geometry of the path with no coordinates ever supplied. Notice also that nodes 1 and 3 share pe_2 but differ in pe_1: it takes **both** coordinates together to separate the symmetric endpoints, which is exactly why we keep several eigenvectors rather than one.

9 Summary

Concept	What to remember
Laplacian L	$D - A$; its quadratic form $x^\top L x = \sum_{ij \in E} \left(x_i - x_j\right)^2$ measures roughness
Eigenvectors	the graph's natural vibration modes, ordered smooth $\rightarrow$ rough by eigenvalue
Drop u_1	the constant mode (eigenvalue 0) carries no position; multiplicity = # components
L_{sym}	normalized variant used in code; eigenvalues in $[0, 2]$, matches GCN
Token	each node's row across the kept eigenvectors = its structural coordinates
Equivariance	relabel the graph and the encoding permutes with it — structure-only
Ambiguity	sign flips and degenerate eigenspaces; fix with sign-flip augmentation / SignNet

Laplacian eigenvector tokenization is, at heart, one idea: **let the graph tell you where its nodes are, in its own basis of vibration modes,** instead of imposing arbitrary numbers from outside. That is precisely the permutation-aware, structure-only identity the earlier tokenizations were reaching for and missing.